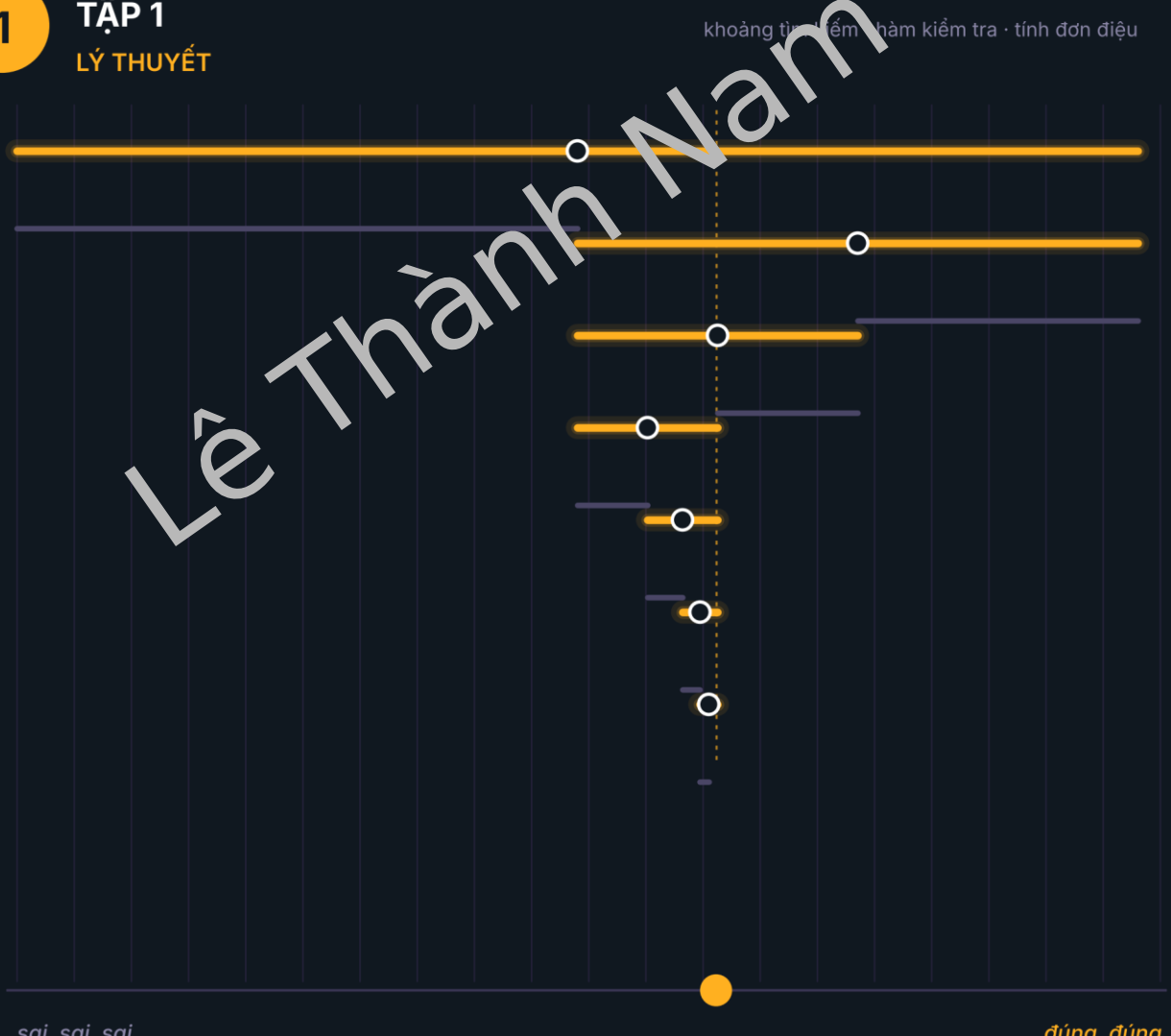
TÀI LIỆU BỒI DƯỠNG HỌC SINH GIỎI TIN HỌC

CHẶT NHỊ PHÂN TRÊN KẾT QUẢ

Binary Search on the Answer

Đoán đáp án, kiểm tra, chia đôi

1 **TẬP 1**
LÝ THUYẾT



Lê Thành Nam

Tổng hợp và biên soạn
Lê Thành Nam

1 Lý thuyết 2 Bài tập 3 Hướng dẫn giải

LỜI NÓI ĐẦU

Hãy nhớ lại trò chơi đoán số quen thuộc: Bạn em nghĩ một số từ 1 tới 100, em đoán, bạn chỉ trả lời "lớn hơn" hoặc "nhỏ hơn". Người chơi giỏi luôn đoán 50 trước, rồi 25 hoặc 75, rồi tiếp tục chia đôi. Chỉ sau tối đa 7 lần đoán, em chắc chắn tìm ra số của bạn. Nếu bạn nghĩ một số tới một tỉ, em cũng chỉ cần 30 lần. Cách đoán "luôn hỏi ở chính giữa" ấy được gọi là chặt nhị phân (binary search).

Hầu hết học sinh gặp chặt nhị phân lần đầu ở bài toán "tìm một số trong mảng đã sắp xếp". Nhưng sức mạnh thật sự của nó nằm ở một ý tưởng sâu hơn nhiều, được gọi là chặt nhị phân trên kết quả: Khi bài toán hỏi "giá trị nhỏ nhất (hoặc lớn nhất) là bao nhiêu", thay vì đi tìm trực tiếp, ta đoán đáp án rồi chỉ trả lời một câu hỏi dễ hơn rất nhiều: "Với giá trị này, có làm được không?". Một bài toán tối ưu khó biến thành khoảng 30 đến 60 lần trả lời một câu hỏi "có hoặc không". Kĩ thuật này xuất hiện trong đề thi học sinh giỏi ở mọi cấp, từ bài dễ nhất tới bài khó nhất, và thường là chìa khoá duy nhất để vượt qua những subtask lớn.

Bộ tài liệu này được viết cho người mới hoàn toàn với chặt nhị phân. Em chỉ cần biết lập trình C++ cơ bản: Mảng, vector, vòng lặp, hàm và hàm sort. Những công cụ khác như tổng tiền tố, hai con trỏ, BFS, Dijkstra hay cây Fenwick đều được giới thiệu lại vừa đủ ở chỗ cần dùng. Bộ sách gồm ba tập đi cùng nhau:

- Tập 1: Lý thuyết (cuốn sách này) gồm 12 bài học chia thành 4 chương: Nền tảng; Hàm kiểm tra tham lam và đếm; Hàm kiểm tra phức tạp; Nâng cao.
- Tập 2: Bài tập gồm 36 bài, mỗi bài học có ba bài tập ở ba mức độ (cơ bản, trung bình, nâng cao), có ràng buộc theo subtask như đề thi thật.
- Tập 3: Hướng dẫn giải phân tích từng bài tập từ cách ngây thơ đến cách tối ưu, kèm chương trình hoàn chỉnh và bảng điểm của từng cách làm.

Mỗi bài học không đưa ra thuật toán ngay. Em sẽ được đặt trước một bài toán cụ thể, thử cách ngây thơ, đếm số phép tính để thấy chỗ bế tắc, rồi từng bước khám phá ra cách chặt nhị phân. Trọng tâm của mọi bài là hai câu hỏi: Hàm kiểm tra là gì, và vì sao nó đơn điệu (một khi đã "làm được" thì nới lỏng thêm vẫn "làm được")? Mọi ví dụ đều được chạy tay bằng bảng hoặc hình vẽ để em tự kiểm tra từng bước.

Mọi chương trình trong sách đều đã được kiểm tra bằng cách đối chiếu với lời giải vét cạn trên hàng trăm bộ dữ liệu ngẫu nhiên, gồm cả những bộ "xấu" như mọi phần tử bằng nhau, đáp án nằm sát biên, không có đáp án; các chương trình tối ưu đều đã được chạy trên dữ liệu lớn nhất để bảo đảm thời gian.

Có một nguyên tắc xuyên suốt cả bộ sách: Chặt nhị phân chỉ có ba chỗ dễ sai: Khoảng tìm kiếm, điều kiện dừng và hàm kiểm tra. Sách dùng một khuôn mẫu duy nhất, được giải thích kĩ ở Bài 1 và Bài 3, để em không bao giờ phải băn khoăn "lo = mid hay lo = mid + 1", "while (lo < hi) hay while (lo <= hi)" nữa.

Chúc các em học tập say mê và đạt kết quả cao!

Lê Thành Nam

Lê Thành Nam

CÁCH HỌC VỚI TÀI LIỆU NÀY

Mỗi bài học đi theo cùng một con đường, giống như cách một người giỏi thật sự suy nghĩ khi gặp bài mới:

Phần	Nội dung
Khởi động	Một câu hỏi nhỏ, gần gũi để em bắt đầu nghĩ.
Bài toán	Đề bài cụ thể, có ví dụ và hình minh họa.
Ý tưởng ngây thơ	Cách làm đầu tiên ai cũng nghĩ ra (thường là thử lần lượt từng đáp án), kèm phần đếm số phép tính.
Bế tắc	Chỉ ra chính xác vì sao cách ngây thơ chậm, hoặc vì sao một cách làm "nghe có lí" lại sai.
Phá bế tắc	Từng bước: Hàm kiểm tra, tính đơn điệu, khoảng tìm kiếm, chạy tay, chương trình.
Mở rộng	Các biến thể thường gặp trong đề thi.
Bẫy thường gặp	Những lỗi đã làm mất điểm của nhiều thế hệ học sinh.
Ghi nhớ	Tóm tắt những điều cốt lõi nhất.
Luyện tập	Liên hệ tới các bài tập ở Tập 2 và vài câu tự kiểm tra có đáp số.

Các khung màu trong sách có ý nghĩa cố định:

Khung	Ý nghĩa
KHỞI ĐỘNG	Câu hỏi mở đầu bài học.
ĐỊNH NGHĨA, ĐỊNH LÍ	Khái niệm mới, phát biểu chính xác.
CÔNG THỨC	Khuôn chương trình hoặc công thức then chốt của bài.
ĐẾM SỐ PHÉP TÍNH	Phân tích độ phức tạp và giới hạn của các con số (có tràn số hay không).
BẾ TẮC	Chỗ cách làm hiện tại không vượt qua được, hoặc một phản ví dụ.
VÌ SAO ĐÚNG?	Lập luận chứng minh hàm kiểm tra đúng và đơn điệu.
BẪY THƯỜNG GẶP	Các lỗi hay gặp.
GHI NHỚ	Tóm tắt cuối bài.
ĐỌC THÊM	Nội dung mở rộng dành cho học sinh muốn tìm hiểu sâu hơn.

Bốn câu hỏi trước mọi bài chặt nhị phân trên kết quả. Khi nghi ngờ một bài có thể chặt nhị phân, hãy tập thói quen tự hỏi:

1. **Đáp án là gì, nằm trong khoảng nào?** Viết ra giá trị nhỏ nhất và lớn nhất mà đáp án có thể nhận. Khoảng này phải **chắc chắn** chứa đáp án.

2. **Hàm kiểm tra là gì?** Phát biểu câu hỏi "có hoặc không" ok(X): Ví dụ "trong X giây có in xong không?", "mỗi người chép không quá X trang thì có đủ người không?".

Lê Thành Nam

3. **Hàm kiểm tra có đơn điệu không?** Nếu ok(X) đúng thì ok(X + 1) có chắc đúng không (hoặc ngược lại)? Đây là điều kiện **bắt buộc**, phải lập luận được.

4. **Tính ok(X) thế nào cho nhanh?** Thường là tham lam, đếm, hai con trỏ, BFS hay quy hoạch động, với độ phức tạp đủ nhỏ để nhân thêm khoảng 30 đến 60 lần.

Khuôn chương trình dùng xuyên suốt sách. Mọi bài đều quy về một trong hai dạng, khác nhau đúng ở một chỗ:

	Tìm X nhỏ nhất **có ok(X)**	**Tìm X** lớn nhất **có ok(X)**
Dạng ok	sai, sai, ..., sai, **đúng**, đúng, ...	đúng, ..., **đúng**, sai, sai, ...
Tính mid	lo + (hi − lo) / 2	lo + (hi − lo + 1) / 2
Khi ok(mid)	hi = mid	lo = mid
Khi không ok(mid)	lo = mid + 1	hi = mid − 1

Vòng lặp luôn là while (lo < hi), và khi dừng thì lo = hi chính là đáp án. Bài 1 giải thích vì sao khuôn này không bao giờ lặp vô hạn và không bao giờ bỏ sót đáp án.

Lời khuyên khi học: Hãy đọc với giấy và bút bên cạnh. Mỗi khi gặp một bảng chạy tay, hãy che phần đáp án lại và tự điền trước. Mỗi khi gặp một hàm kiểm tra, hãy tự hỏi "vì sao nó đơn điệu?" trước khi đọc lời giải thích. Chỉ khi tự viết lại được chương trình và tự giải thích được vì sao nó đúng, em mới thật sự nắm được bài.

Bản đồ đọc sách. Chương 1 là nền móng: Chặt nhị phân trên mảng, trên một hàm số, và lần đầu tiên chặt nhị phân trên kết quả. Chương 2 tập trung vào loại hàm kiểm tra phổ biến nhất trong đề thi: Tham lam (chia dãy, đặt điểm) và đếm (tìm phần tử thứ k). Chương 3 đưa hàm kiểm tra lên một tầng mới: Hai con trỏ, duyệt đồ thị, và chặt nhị phân trên số thực. Chương 4 dạy những biến đổi khéo léo: Biến bài toán trung bình cộng thành bài toán tổng, kết hợp với tổng tiền tố hai chiều và BFS đa nguồn.

Lê Thành Nam

MỤC LỤC

CHƯƠNG 1. NỀN TẢNG 6
Bài 1. Chặt nhị phân trên mảng đã sắp xếp 7
Bài 2. Chặt nhị phân trên một hàm số: Căn bậc k 15
Bài 3. Chặt nhị phân trên kết quả: Bài toán máy in 21

CHƯƠNG 2. HÀM KIỂM TRA THAM LAM VÀ ĐẾM 28
Bài 4. Cực tiểu hoá giá trị lớn nhất: Chép sách 29
Bài 5. Cực đại hoá giá trị nhỏ nhất: Đặt bộ phát wifi 36
Bài 6. Phần tử nhỏ thứ k: Bảng cửu chương 42

CHƯƠNG 3. HÀM KIỂM TRA PHỨC TẠP 48
Bài 7. Đếm cặp bằng hai con trỏ: Khoảng cách nhỏ thứ k 49
Bài 8. Hàm kiểm tra là một lần duyệt đồ thị: Xe tải qua cầu 55
Bài 9. Chặt nhị phân trên số thực: Cắt dây 62

CHƯƠNG 4. NÂNG CAO 68
Bài 10. Trung bình cộng lớn nhất: Chặt nhị phân trên tỉ số 69
Bài 11. Chặt nhị phân trên bảng: Hình vuông lớn nhất 75
Bài 12. Thời gian trong hàm kiểm tra: Thoát khỏi đám cháy 81

Lê Thành Nam

CHƯƠNG 1. NỀN TẢNG

*Chương đầu tiên xây nền móng cho cả cuốn sách. Bài 1 dạy chặt nhị phân trên mảng đã sắp xếp và một khuôn chương trình duy nhất không bao giờ lặp vô hạn. Bài 2 chuyển từ mảng sang một hàm số: Tìm căn bậc k của một số rất lớn mà không cần mảng nào cả. Bài 3 là bước ngoặt: Lần đầu tiên ta chặt nhị phân trên chính **đáp án** của bài toán. Ba bài học này là nền móng cho mọi chương sau, nên hãy đọc thật chậm.*

Lê Thành Nam

BÀI 1 Chặt nhị phân trên mảng đã sắp xếp

Học xong bài này, em sẽ:

✓ Hiểu vì sao chặt nhị phân chỉ cần khoảng $\log_2 n$ bước để tìm trong n phần tử.

✓ Nhìn một bài tìm kiếm dưới dạng "dãy sai, sai, ..., sai, đúng, đúng, ..., đúng" và tìm điểm chuyển.

✓ Thuộc khuôn chương trình while (lo < hi) và giải thích được vì sao nó dừng và không bỏ sót đáp án.

✓ Dùng được lower_bound, upper_bound của thư viện và biết tự viết lại chúng.

KHỞI ĐỘNG

Bạn em nghĩ một số nguyên từ 1 đến 100. Mỗi lần em đoán, bạn trả lời "đúng rồi", "số của tớ lớn hơn" hoặc "số của tớ nhỏ hơn". Em có chắc tìm được số ấy sau **7 lần đoán** không? Còn nếu bạn nghĩ một số từ 1 đến một tỉ thì cần bao nhiêu lần?

1. Bài toán TIMSO

BÀI TOÁN. TÌM SỐ [TIMSO]

Cho dãy gồm n số nguyên $a_1, a_2, ..., a_n$ (chưa được sắp xếp). Có q câu hỏi, mỗi câu hỏi cho hai số $l \le r$: Hãy đếm xem trong dãy có bao nhiêu phần tử có giá trị nằm trong đoạn [l, r].

Dữ liệu vào: Dòng đầu gồm hai số n, q ($1 \le n, q \le 2 \cdot 10^5$). Dòng thứ hai gồm n số a_i ($|a_i| \le 10^9$). Mỗi dòng trong q dòng tiếp theo gồm hai số l, r ($|l|, |r| \le 10^9, l \le r$).

Kết quả: Với mỗi câu hỏi, in ra một dòng gồm số phần tử có giá trị trong [l, r].

Dữ liệu vào	Kết quả	Giải thích
8 3 5 1 9 3 7 3 12 8 3 8 4 4 1 12	5 0 8	Đoạn [3, 8] chứa các giá trị 3, 3, 5, 7, 8: có 5 phần tử. Không có phần tử nào bằng 4. Mọi phần tử đều nằm trong [1, 12].

2. Ý tưởng ngây thơ: Duyệt cả dãy cho mỗi câu hỏi

Cách tự nhiên nhất: Với mỗi câu hỏi, duyệt lần lượt n phần tử và đếm những phần tử có $l \le a_i \le r$.

Đếm trực tiếp

```
for (int i = 0; i < n; i++)
    if (l <= a[i] && a[i] <= r) cnt++;
```

Lê Thành Nam

ĐẾM SỐ PHÉP TÍNH

Mỗi câu hỏi tốn n phép so sánh, cả bài tốn $n \cdot q = 2 \cdot 10^5 \cdot 2 \cdot 10^5 = 4 \cdot 10^{10}$ phép tính. Một máy chấm thông thường làm được khoảng 10^8 đến 10^9 phép tính mỗi giây, nên cách này chạy mất hàng phút.

BẪY TẮC

Mỗi câu hỏi đều phải "nhìn" lại toàn bộ dãy, dù các câu hỏi chỉ khác nhau ở l và r. Ta cần sắp xếp lại dữ liệu một lần để sau đó mỗi câu hỏi được trả lời mà **không cần nhìn hết** các phần tử.

3. Sắp xếp và đổi câu hỏi

Sau khi sắp xếp tăng dần, dãy ví dụ trở thành

$$a = 1, 3, 3, 5, 7, 8, 9, 12 \quad \text{(chỉ số 0, 1, ..., 7)}$$

Các phần tử có giá trị trong [3, 8] nằm **liền nhau** thành một đoạn: 3, 3, 5, 7, 8 (chỉ số 1 tới 5). Để đếm chúng, ta chỉ cần biết hai vị trí:

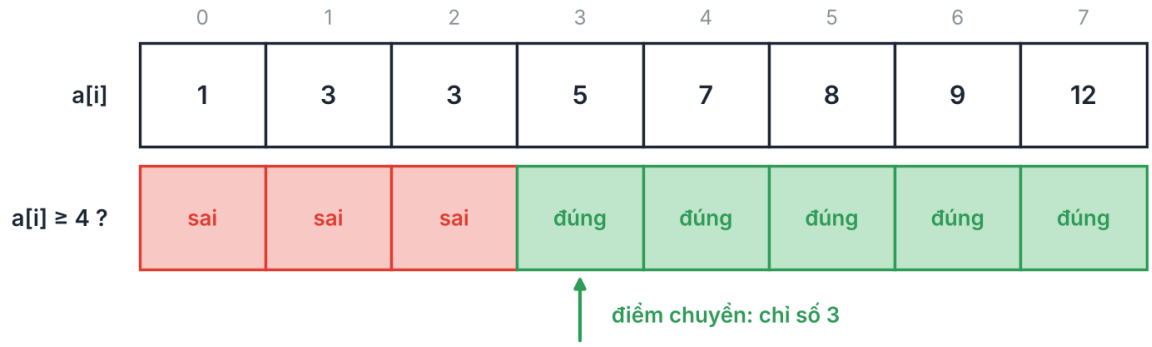
- **firstGE(l)**: vị trí đầu tiên có a[i] ≥ l. Với l = 3, đó là chỉ số 1.
- **firstGE(r + 1)**: vị trí đầu tiên a[i] ≥ r + 1, tức a[i] > r. Với r = 8, chỉ số 6 (giá trị 9).

Các phần tử từ chỉ số firstGE(l) tới trước firstGE(r + 1) chính là những phần tử cần đếm, nên đáp số là **firstGE(r + 1) − firstGE(l)** = 6 − 1 = 5. Nếu mọi phần tử đều nhỏ hơn x thì quy ước firstGE(x) = n.

Bài toán bây giờ chỉ còn **Tìm nhanh vị trí đầu tiên có a[i] ≥ x** trong một mảng đã sắp xếp.

4. Dãy sai rồi đúng

Hãy viết cạnh mỗi phần tử câu trả lời cho câu hỏi "a[i] ≥ x?". Với x = 4:



Hình 1.1. *Vì mảng đã sắp xếp, các câu trả lời luôn có dạng sai, ..., sai, đúng, ..., đúng. Ta cần tìm ô "đúng" đầu tiên.*

Vì mảng tăng dần, một khi a[i] ≥ x thì mọi phần tử đứng sau cũng ≥ x. Do đó dãy câu trả lời luôn là một khối **sai** liền nhau rồi tới một khối **đúng** liền nhau (một trong hai khối có thể rỗng). Ta gọi một dãy như vậy là **đơn điệu**.

Lê Thành Nam

ĐỊNH NGHĨA

Một điều kiện ok(i) trên các chỉ số 0, 1, ..., n − 1 được gọi là **đơn điệu** nếu ok(i) đúng kéo theo ok(i + 1) đúng. Khi đó các giá trị ok(0), ok(1), ..., ok(n − 1) có dạng **sai, ..., sai, đúng, ..., đúng**, và chỉ số đầu tiên làm ok đúng được gọi là **điểm chuyển**.

Muốn tìm điểm chuyển, ta không cần hỏi từng ô. Hãy hỏi **ô chính giữa**:

- Nếu ô giữa là **đúng**, điểm chuyển nằm ở ô giữa hoặc bên trái nó: bỏ hẳn nửa phải.
- Nếu ô giữa là **sai**, điểm chuyển nằm hẳn bên phải ô giữa: bỏ ô giữa và cả nửa trái.

Mỗi lần hỏi bỏ đi một nửa số ô còn nghi ngờ. Đó chính là chiến thuật của người chơi đoán số.

5. Khuôn chương trình

Ta giữ hai biến lo, hi với một **bất biến** (điều luôn đúng ở mọi vòng lặp): **Đáp án nằm trong đoạn [lo, hi]**. Ban đầu lo = 0, hi = n (đáp án n nghĩa là "không có phần tử nào ≥ x").

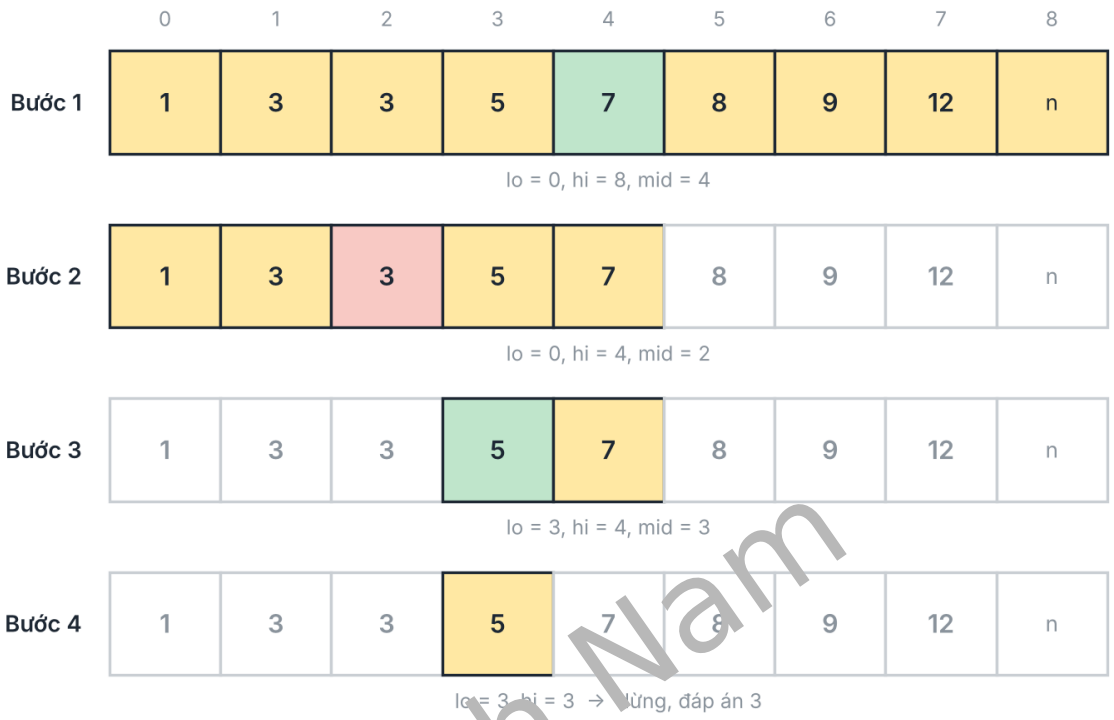
Tìm vị trí đầu tiên có a[i] >= x

```
int firstGE(ll x) {
    int lo = 0, hi = n;                 // đáp án nằm trong [lo, hi]
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;   // lo <= mid < hi
        if (a[mid] >= x) hi = mid;      // đáp án ở mid hoặc bên trái
        else lo = mid + 1;              // đáp án ở bên phải mid
    }
    return lo;                          // lo == hi
}
```

Chạy tay với a = 1, 3, 3, 3, 7, 8, 9, 12 và x = 4:

lo	hi	mid	a[mid]	a[mid] ≥ 4?	Việc làm
0	8	4	7	đúng	hi = 4
0	4	2	3	sai	lo = 3
3	4	3	5	đúng	hi = 3
3	3				dừng, trả về 3

Lê Thành Nam



Hình 1.2. *Chặt nhị phân tìm vị trí đầu tiên có a[i] ≥ 4. Vùng vàng là đoạn [lo, hi] còn nghi ngờ; ô mid tô xanh nếu a[mid] ≥ 4, tô đỏ nếu ngược lại.*

Lê Thành Nam

VÌ SAO ĐÚNG?

Khuôn chương trình luôn dừng và luôn đúng.

1. Bất biến "đáp án nằm trong [lo, hi]" được giữ: Khi a[mid] ≥ x, đáp án ≤ mid nên gán hi = mid không làm mất đáp án. Khi a[mid] < x, mọi chỉ số ≤ mid đều "sai" nên đáp án ≥ mid + 1.
2. Vì lo < hi nên mid = lo + (hi − lo)/2 thoả lo ≤ mid < hi. Cả hai nhánh hi = mid và lo = mid + 1 đều làm đoạn [lo, hi] **ngắn đi ít nhất 1**, nên vòng lặp chắc chắn dừng.
3. Khi dừng, lo = hi và đáp án nằm trong [lo, lo], tức đáp án chính là lo.

ĐẾM SỐ PHÉP TÍNH

Mỗi vòng lặp độ dài đoạn [lo, hi] giảm còn khoảng một nửa. Bắt đầu từ $n = 2 \cdot 10^5$, sau 18 vòng đoạn chỉ còn 1 phần tử (vì $2^{18} = 262144 > 2 \cdot 10^5$). Mỗi câu hỏi gọi firstGE hai lần, nên cả bài chỉ tốn khoảng $n \cdot \log_2 n$ cho việc sắp xếp và $2 \cdot q \cdot 18$ phép so sánh: Dưới 10^7 phép tính.

CÔNG THỨC

Hai dạng chặt nhị phân. Với một điều kiện ok đơn điệu trên các số nguyên:

- Tìm X **nhỏ nhất** có ok(X) (dạng sai, ..., sai, đúng, ...): mid = lo + (hi − lo)/2; nếu ok(mid) thì hi = mid, ngược lại lo = mid + 1.
- Tìm X **lớn nhất** có ok(X) (dạng đúng, ..., đúng, sai, ...): mid = lo + (hi − lo + 1)/2; nếu ok(mid) thì lo = mid, ngược lại hi = mid − 1.

Cả hai đều dùng while (lo < hi) và trả về lo.

Vì sao dạng thứ hai phải cộng thêm 1 khi tính mid? Hãy thử lo = 3, hi = 4 với cách tính mid = lo + (hi − lo)/2 = 3. Nếu ok(3) đúng ta gán lo = mid = 3: đoạn [3, 4] **không thay đổi**, và vòng lặp chạy mãi mãi. Cộng thêm 1 làm mid = 4 > lo, nên nhánh lo = mid luôn làm đoạn ngắn lại.

6. Chương trình hoàn chỉnh

TIMSO.CPP: Sắp xếp và chặt nhị phân, O((n + q) log n)

```cpp
#include <algorithm>
#include <cstdio>
#include <iostream>
#include <vector>
using namespace std;
typedef long long ll;

int n;
vector<ll> a;

// Vị trí đầu tiên có a[i] >= x (bằng n nếu không có)
int firstGE(ll x) {
    int lo = 0, hi = n;                 // đáp án nằm trong [lo, hi]
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        // mid thoả mãn: đáp án ở mid hoặc bên trái
        if (a[mid] >= x) hi = mid;
        else lo = mid + 1;              // mid không thoả: đáp án ở bên phải mid
    }
    return lo;
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    if (fopen("TIMSO.INP", "r")) {
        freopen("TIMSO.INP", "r", stdin);
        freopen("TIMSO.OUT", "w", stdout);
    }
    int q;
    cin >> n >> q;
    a.resize(n);
    for (int i = 0; i < n; i++) cin >> a[i];
    sort(a.begin(), a.end());
    while (q--) {
        ll l, r;
        cin >> l >> r;
        // số phần tử >= l trừ đi số phần tử >= r + 1
        cout << firstGE(r + 1) - firstGE(l) << '\n';
    }
    return 0;
}
```

Lê Thành Nam

ĐẾM SỐ PHÉP TÍNH

Các giá trị a_i nằm trong int, nhưng r + 1 có thể bằng $10^9 + 1$; chương trình dùng long long cho l, r để khỏi phải lo nghĩ. Phép tính mid = lo + (hi − lo)/2 thay cho (lo + hi)/2 là một thói quen tốt: Khi lo, hi là những số rất lớn (ở các bài sau, hi có thể tới 10^{18}), lo + hi có thể vượt quá giới hạn của kiểu số.

7. Thư viện chuẩn: lower_bound và upper_bound

C++ đã có sẵn hai hàm làm đúng việc của firstGE trên một vector (hoặc mảng) đã sắp xếp:

Hàm	Trả về
lower_bound(a.begin(), a.end(), x)	Vị trí đầu tiên có a[i] ≥ x
upper_bound(a.begin(), a.end(), x)	Vị trí đầu tiên có a[i] > x

Hai hàm trả về một con trỏ (iterator); trừ đi a.begin() để được chỉ số. Đáp số của TIMSO viết gọn thành:

Đếm bằng thư viện

```
cout << (upper_bound(a.begin(), a.end(), r) - a.begin())
      - (lower_bound(a.begin(), a.end(), l) - a.begin()) << '\n';
```

Sách vẫn dạy em tự viết firstGE vì từ Bài 2 trở đi, ta sẽ chặt nhị phân trên những "dãy" **không hề được lưu trong bộ nhớ**: các số từ 1 tới 10^{18}, các mức thời gian, các độ dài. Khi đó không có hàm thư viện nào giúp được, chỉ có khuôn chương trình ở mục 5.

8. Mở rộng

Mở rộng 1: Đếm số lần xuất hiện

Số phần tử bằng đúng x là upper_bound(x) − lower_bound(x): Những phần tử bằng x nằm liền nhau trong mảng đã sắp xếp (Bài tập 1.1, DEMSO).

Mở rộng 2: Phần tử gần nhất

Phần tử gần x nhất chỉ có thể là a[i] (phần tử nhỏ nhất ≥ x) hoặc a[i − 1] (phần tử lớn nhất < x), với i = lower_bound(x). Chỉ cần so sánh hai ứng viên, nhớ xử lí khi i = 0 hoặc i = n (Bài tập 1.2, GANNHAT).

Mở rộng 3: Đếm cặp

Đếm số cặp i < j có L ≤ $a_i + a_j$ ≤ R: Sau khi sắp xếp, với mỗi i, các j hợp lệ thoả L − a_i ≤ a_j ≤ R − a_i, chính là bài TIMSO trên đoạn a[i + 1 .. n − 1] (Bài tập 1.3, TONGCAP).

Lê Thành Nam

BẪY THƯỜNG GẶP

- Quên sắp xếp trước khi chặt nhị phân: Chương trình vẫn chạy, vẫn in ra số, nhưng sai.
- Viết mid = lo + (hi − lo)/2 cùng với lo = mid: Lặp vô hạn khi hi = lo + 1.
- Đặt hi = n − 1 thay vì n khi đáp án có thể là "không có phần tử nào": Câu hỏi với x lớn hơn mọi phần tử sẽ trả lời sai.
- Viết (lo + hi)/2 khi lo, hi gần 10^{18}: Tràn số.
- Dùng while (lo <= hi) trộn với hi = mid: Một kiểu khuôn khác, rất dễ lặp vô hạn. Hãy chỉ dùng **một** khuôn duy nhất.

GHI NHỚ

- Chặt nhị phân tìm **điểm chuyển** của một dãy dạng sai, ..., sai, đúng, ..., đúng (hoặc ngược lại).
- Giữ bất biến "đáp án nằm trong [lo, hi]"; khi lo = hi thì đó là đáp án.
- Tìm nhỏ nhất: hi = mid / lo = mid + 1. Tìm lớn nhất: lo = mid / hi = mid − 1 và mid làm tròn **lên**.
- Số bước khoảng $\log_2$ của độ dài khoảng tìm kiếm: 18 bước cho $2 \cdot 10^5$, 30 bước cho 10^9, 60 bước cho 10^{18}.

Luyện tập

Bài	Tên	Mức độ	Gợi ý liên hệ
1.1	DEMSO	Cơ bản	Mở rộng 1: upper_bound trừ lower_bound.
1.2	GANNHAT	Trung bình	Mở rộng 2: Hai ứng viên quanh lower_bound.
1.3	TONGCAP	Nâng cao	Mở rộng 3: Đếm cặp bằng chặt nhị phân trên từng i.

Tự kiểm tra nhanh

1. Chặt nhị phân trên một mảng 10^6 phần tử cần nhiều nhất bao nhiêu vòng lặp? *(Đáp số: 20 vòng, vì $2^{20} = 1048576 > 10^6$.)*

2. Với a = 2, 4, 4, 4, 9, các giá trị lower_bound(4) và upper_bound(4) (theo chỉ số) là bao nhiêu? *(Đáp số: 1 và 4; có 3 số 4.)*

3. Trong dạng "tìm lớn nhất", nếu lo = 6, hi = 7 thì mid bằng bao nhiêu? *(Đáp số: mid = 6 + (7 − 6 + 1)/2 = 7.)*

4. Trò chơi đoán số từ 1 tới một tỉ cần nhiều nhất bao nhiêu lần đoán? *(Đáp số: 30 lần, vì $2^{30} > 10^9$.)*

Lê Thành Nam